

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

Họ và tên: Lê Thành Nguyên

MSSV: 20236049

Môn học: Project I (IT3150)

Chủ đề: Bài toán người giao hàng (TSP)

Giảng viên hướng dẫn: Mai Xuân Ngọc

Chương 1: Giới thiệu chung

1. Giai đoạn 1: Lập trình cơ bản

- Week 1 + 2: Ôn tập về tính toán, các bài toán trên mảng, bài toán xử lý chuỗi ký tự đơn giản. Đồng thời luyện tập các thuật toán và kỹ thuật lập trình đơn giản như fibonacci, hoán vị,...
- Week 3 + 4: Ôn tập các cấu trúc dữ liệu quan trọng trong lập trình như ngăn xếp, hàng đợi, cây nhị phân, danh sách liên kết. Ngoài ra cũng luyện tập các bài toán nâng cao hơn với chuỗi ký tự.
- Week 5 + 6: Ôn tập các dạng toán thường gặp với các giải thuật như BFS, DFS, Kruskal, Hamilton,... trên đồ thị có hướng và vô hướng.
- Week 7 + 8: Ôn tập các bài tập kết hợp xử lý tính toán và chuỗi ký tự, điển hình như các bài chuyển tiền trong tài khoản ngân hàng, kiểm soát dữ liệu dân cư,...

Các ngôn ngữ lập trình để giải quyết bài toán: C/C++, Java.

Các IDE sử dụng: Dev C++, Eclipse.

2. Giai đoạn 2: Xây dựng ứng dụng đơn giản

Trong Project I, sinh viên lựa chọn nghiên cứu và cài đặt bài toán người giao hàng (Travelling Salesman Problem – TSP), một bài toán kinh điển trong lĩnh vực tối ưu hóa tổ hợp và thuật toán.

Bài toán TSP đặt ra yêu cầu tìm một lộ trình ngắn nhất để người giao hàng xuất phát từ một điểm, đi qua tất cả các địa điểm cần giao hàng đúng một lần và quay trở lại điểm xuất phát ban đầu.

Thông qua việc giải quyết bài toán này, project tập trung rèn luyện các kỹ năng lập trình cơ bản, tư duy thuật toán, cũng như làm quen với việc xây dựng ứng dụng giải quyết bài toán thực tế, từ khâu phân tích yêu cầu, thiết kế thuật toán cho đến cài đặt và kiểm thử chương trình.

Ngôn ngữ lập trình triển khai: Java cho Backend và HTML/CSS/JavaScript cho Frontend

Chương 2: Lập trình cơ bản

1. Week 1 + 2

Ở 2 tuần đầu, sinh viên được khởi động với các bài toán khá quen thuộc và đơn giản.

Ví dụ 1: Cho một thời điểm được biểu diễn dưới dạng chuỗi theo định dạng hh:mm:ss, trong đó:

- hh ($0 \leq hh \leq 23$) là giờ,
- mm ($0 \leq mm \leq 59$) là phút,
- ss ($0 \leq ss \leq 59$) là giây.

Hãy chuyển thời điểm này sang số giây, theo công thức:
kết quả = $hh \times 3600 + mm \times 60 + ss$.

Input: Dòng 1: chứa một chuỗi s biểu diễn thời điểm.

Output

- Nếu s không đúng định dạng hh:mm:ss, hãy in ra INCORRECT.
- Ngược lại, in ra giá trị sau khi chuyển đổi sang giây.

Ví dụ

Input: 13:05:26

Output: 47126

Các bước giải:

- Kiểm tra xem độ dài chuỗi có hợp lệ không, nếu khác 8 thì không hợp lệ.
- Kiểm tra các ký tự ở chỉ số 2 và 5 có hợp lệ không.
- Tách chuỗi con từ chuỗi input để lấy giờ, phút, giây riêng biệt.
- Kiểm tra xem các ký tự ở chuỗi được tách ra có hợp lệ không
- Chuyển các chuỗi ký tự thành số nguyên và kiểm tra hợp lệ.
- Tính tổng theo công thức.

Mã nguồn C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    if (s.size() != 8) {
```

```
    cout << "INCORRECT";
    return 0;
}
if (s[2] != ':' || s[5] != ':') {
    cout << "INCORRECT";
    return 0;
}
string hh = s.substr(0, 2);
string mm = s.substr(3, 2);
string ss = s.substr(6, 2);
if (!isdigit(hh[0]) || !isdigit(hh[1]) ||
    !isdigit(mm[0]) || !isdigit(mm[1]) ||
    !isdigit(ss[0]) || !isdigit(ss[1])) {
    cout << "INCORRECT";
    return 0;
}
int h = stoi(hh);
int m = stoi(mm);
int sec = stoi(ss);
if (h < 0 || h > 23 || m < 0 || m > 59 || sec < 0 || sec > 59) {
    cout << "INCORRECT";
    return 0;
}
cout << h * 3600 + m * 60 + sec;
}
```

Ví dụ 2: Cho một số nguyên n , hãy viết chương trình sinh ra tất cả các dãy nhị phân có độ dài n theo thứ tự từ điển (lexicographic order).

Input

Dòng 1: chứa một số nguyên n ($1 \leq n \leq 20$)

Output

In ra các dãy nhị phân theo thứ tự từ điển, mỗi dãy trên một dòng.

Các bước giải:

- Áp dụng thuật toán đệ quy quay lui để sinh ra dãy nhị phân
- Sử dụng vòng lặp để xét các trường hợp của từng phần tử (0 và 1)
- Nếu dãy đã đủ số phần tử, in ra dãy. Nếu chưa đủ, tiếp tục gọi đệ quy để gán giá trị

Mã nguồn C++:

```
#include<bits/stdc++.h>
using namespace std;
int n;
int a[100];
void print(){
    for(int i = 1; i <= n; i++){
        cout << a[i];
    }
    cout << endl;
}
void Try(int i){
    for(int j = 0; j <= 1; j++){
        a[i] = j;
        if(i == n){
            print();
        }
        else{
            Try(i + 1);
        }
    }
}
int main(){
    cin >> n;
    Try(1);
}
```

2. Week 3 + 4

Ở giai đoạn này, sinh viên được ôn tập các cấu trúc dữ liệu quan trọng trong lập trình: ngăn xếp, hàng đợi, danh sách liên kết, cây nhị phân, kỹ thuật lập trình và chuỗi ký tự (nâng cao)

Ví dụ 1: Thực hiện một chuỗi các thao tác trên ngăn xếp (stack), trong đó mỗi phần tử là một số nguyên:

- PUSH v: đưa (đẩy) giá trị v vào ngăn xếp
- POP: lấy một phần tử ra khỏi ngăn xếp và in giá trị đó ra stdout (in NULL nếu ngăn xếp đang rỗng)

Input

Mỗi dòng chứa một lệnh (thao tác) thuộc một trong hai dạng:

PUSH v

POP

Output: In ra kết quả của các thao tác POP, mỗi kết quả trên một dòng.

Ví dụ:

Input:

PUSH 1

PUSH 2

PUSH 3

POP

POP

PUSH 4

PUSH 5

POP

#

Output:

3

2

5

Các bước giải:

- Khởi tạo một ngăn xếp
- Nếu gặp lệnh PUSH, đẩy phần tử mới vào ngăn xếp và in ra
- Nếu gặp lệnh POP, cần kiểm tra ngăn xếp còn phần tử nào không để tránh gặp lỗi

Mã nguồn C++:

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    stack<int> st;
    string s;
```

```
while(cin >> s && s != "#"){
    if(s == "PUSH"){
        int a;
        cin >> a;
        st.push(a);
    }
    else if(s == "POP"){
        if(st.empty()){
            cout << "NULL" << endl;
        }
        else{
            cout << st.top() << endl;
            st.pop();
        }
    }
    else{
        break;
    }
}
```

Ví dụ 2: Cho dãy $a_1, a_2, \dots, a_n$ trong đó các phần tử đôi một khác nhau và 1 giá trị nguyên dương M . Hãy đếm số Q các cặp (i, j) sao cho $1 \leq i < j \leq n$ và $a_i + a_j = M$.

Dữ liệu

- Dòng 1: ghi n và M ($1 \leq n, M \leq 1000000$)
- Dòng 2: ghi $a_1, a_2, \dots, a_n$

Kết quả: Ghi ra giá trị Q

Ví dụ: Dữ liệu

5 6

5 2 1 4 3

Kết quả: 2

Các bước giải:

- Nhập dữ liệu cho mảng và sắp xếp mảng
- Sử dụng hai con trỏ $l = 0$ và $r = n - 1$
- Nếu tổng hai phần tử ở hai chỉ số l và r nhỏ hơn M , tăng l lên 1 đơn vị. Ngược lại, nếu nhỏ hơn thì giảm r 1 đơn vị. Nếu bằng M thì tăng biến kết quả 1 đơn vị và tăng l lên 1 đơn vị.

Mã nguồn C++:

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n, sum;
    cin >> n >> sum;
    int res = 0;
    int a[n];
    for(int i = 0; i < n; i++){
        cin >> a[i];
    }
    sort(a, a + n);
    int l = 0, r = n - 1;
    while(l < r){
        if(a[l] + a[r] == sum){
            res++;
            l++;
        }
        else if(a[l] + a[r] < sum){
            l++;
        }
        else r--;
    }
    cout << res;
}
```

3. Week 5 + 6

Giai đoạn này, sinh viên được ôn tập các dạng toán thường gặp với các giải thuật như BFS, DFS, Kruskal, Hamilton,... trên đồ thị có hướng và vô hướng.

Ví dụ 1: Cho một đồ thị vô hướng $G=(V,E)$, trong đó $V=\{1,2,\dots,n\}$ là tập các đỉnh. Hãy viết chương trình duyệt các đỉnh của đồ thị bằng thuật toán DFS (Depth-First Search), xét theo thứ tự từ điển (lexicographic order) của các đỉnh.

Input

- Dòng 1: chứa hai số nguyên n và m ($1 \leq n, m \leq 100000$)
- Dòng $i+1$: chứa hai số u và v , là hai đỉnh của cạnh thứ i

Output

- In ra dãy các đỉnh được thăm theo thứ tự DFS

Ví dụ: Input

7 12

1 2

1 3

2 3

2 4

2 7

3 5

3 7

4 5

4 6

4 7

5 6

5 7

Output

1 2 3 5 4 6 7

Các bước giải:

- Khởi tạo một mảng các vector để lưu các đỉnh kề nhau, mảng visited để đánh dấu điểm đã được duyệt.
- Sắp xếp thứ tự đỉnh kề với một đỉnh cho trước.
- Thuật toán DFS dựa trên thuật toán quay lui.
- Từ một đỉnh, duyệt tất cả những đỉnh kề với nó theo thứ tự, nếu đỉnh nào đã đi qua thì bỏ qua, nếu chưa đi qua thì coi đó là điểm tiếp tục được xét.

Mã nguồn C++:

```
#include <bits/stdc++.h>
using namespace std;
vector<int> v[100];
int visited[100];
int n, m;
void in(int j){
    for(int i : v[j]){
        if(visited[i] == 0){
            visited[i] = 1;
```

```

        cout << i << " ";
        in(i);
    }
}
int main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int k, l;
        cin >> k >> l;
        v[k].push_back(l);
        v[l].push_back(k);
    }
    for (int i = 1; i <= n; i++) {
        sort(v[i].begin(), v[i].end());
    }
    for (int i = 1; i <= n; i++) {
        if (visited[i] == 0) {
            visited[i] = 1;
            cout << i << " ";
            in(i);
        }
    }
}

```

Ví dụ 2: Cho một đồ thị có hướng $G=(V,E)$, trong đó $V=\{1,2,\dots,n\}$ là tập các đỉnh, và $w(u,v)$ là trọng số (độ dài) của cung (u,v) .

Hãy tính $d(u,v)$ — độ dài đường đi ngắn nhất từ đỉnh u đến đỉnh v trong đồ thị G , với mọi $u,v \in V$.

Input

- Dòng 1: chứa hai số nguyên dương n và m ($1 \leq n, m \leq 10000$)
- Dòng $i+1$ ($i = 1, 2, \dots, m$): chứa ba số nguyên dương u, v, w , trong đó w là trọng số của cung (u,v) ($1 \leq w \leq 1000$)

Output

Dòng i ($i = 1, 2, \dots, n$): ghi hàng thứ i của ma trận d

Nếu không tồn tại đường đi từ đỉnh i đến đỉnh j thì $d(i, j) = -1$

Ví dụ

Input

```

4 9
1 2 9
1 3 7

```

1 4 2
2 1 1
2 4 5
3 4 6
3 2 2
4 1 5
4 2 8

Output

0 9 7 2
1 0 8 3
3 2 0 5
5 8 12 0

Các bước giải:

- Áp dụng thuật toán Floyd - Warshall để tìm đường đi ngắn nhất trên mọi cặp điểm.
- Tạo ma trận $d[i][j]$ để lưu khoảng cách ngắn nhất từ i đến j . Nếu i khác j , khởi tạo khoảng cách là INF (rất lớn), nếu $i = j$ thì khởi tạo bằng 0.
- Cho phép đi qua các đỉnh trung gian để tạo ra các đường đi ngắn hơn:
$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$
- In ra ma trận kết quả đường đi.

Mã nguồn C++:

```
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int d[1005][1005];
int main() {
    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            if(i != j){
                d[i][j] = INF;
            }
            else d[i][j] = 0;
        }
    }
    for(int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        d[u][v] = min(d[u][v], w);
    }
    for(int k = 1; k <= n; k++) {
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= n; j++) {
```

```
        if(d[i][k] < INF && d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    }
}
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= n; j++) {
        if(d[i][j] == INF) cout << -1 << " ";
        else cout << d[i][j] << " ";
    }
    cout << "\n";
}
return 0;
}
```

4. Week 7 + 8

Ở giai đoạn này, sinh viên được làm quen với các bài tập kết hợp xử lý tính toán và chuỗi ký tự, điển hình như các bài chuyển tiền trong tài khoản ngân hàng, kiểm soát dữ liệu dân cư,...

Ví dụ: Dữ liệu về hoạt động bán hàng của một công ty thương mại điện tử (công ty có nhiều cửa hàng) bao gồm một chuỗi các dòng, mỗi dòng (biểu diễn một đơn hàng) có thông tin như sau:

<CustomerID> <ProductID> <Price> <ShopID> <TimePoint>

Trong đó, khách hàng <CustomerID> mua sản phẩm <ProductID> với giá <Price> tại cửa hàng <ShopID> vào thời điểm <TimePoint>.

- <CustomerID>: chuỗi ký tự có độ dài từ 3 đến 10
- <ProductID>: chuỗi ký tự có độ dài từ 3 đến 10
- <Price>: số nguyên dương từ 1 đến 1000
- <ShopID>: chuỗi ký tự có độ dài từ 3 đến 10
- <TimePoint>: chuỗi biểu diễn thời điểm theo định dạng HH:MM:SS (ví dụ: 09:45:20 nghĩa là 9 giờ 45 phút 20 giây)

Yêu cầu

Thực hiện một chuỗi truy vấn với các loại sau:

- ?total_number_orders
→ Trả về tổng số đơn hàng
- ?total_revenue
→ Trả về tổng doanh thu mà công ty thương mại điện tử thu được

- ?revenue_of_shop <ShopID>
→ Trả về tổng doanh thu của cửa hàng <ShopID>
- ?total_consume_of_customer_shop <CustomerID> <ShopID>
→ Trả về tổng số tiền mà cửa hàng <ShopID> bán sản phẩm cho khách hàng <CustomerID>
- ?total_revenue_in_period <from_time> <to_time>
→ Trả về tổng doanh thu của công ty trong khoảng thời gian từ <from_time> đến <to_time> (tính cả hai mốc thời gian)

Input

Dữ liệu vào gồm 2 khối:

- Khối dữ liệu

- Gồm tối đa 100000 dòng
- Mỗi dòng chứa thông tin của một đơn hàng theo định dạng trên
- Khối này kết thúc bằng dòng chứa ký tự #

- Khối truy vấn

- Gồm tối đa 100000 dòng
- Mỗi dòng là một truy vấn như mô tả
- Khối này kết thúc bằng dòng chứa ký tự #

Output

- Với mỗi truy vấn, in ra kết quả tương ứng trên một dòng

Ví dụ

Input

```
C001 P001 10 SHOP001 10:30:10
C001 P002 30 SHOP001 12:30:10
C003 P001 40 SHOP002 10:15:20
C001 P001 80 SHOP002 08:40:10
C002 P001 130 SHOP001 10:30:10
C002 P001 160 SHOP003 11:30:20
#
?total_number_orders
?total_revenue
?revenue_of_shop SHOP001
```

```
?total_consume_of_customer_shop C001 SHOP001
?total_revenue_in_period 10:00:00 18:40:45
#
```

Output

```
6
450
170
40
370
```

Các bước giải:

- Với mỗi dòng dữ liệu được nhập vào, dễ dàng tính được tổng số đơn hàng, tổng doanh thu của công ty thương mại điện tử, tổng doanh thu của cửa hàng và doanh thu tổng số tiền của cửa hàng bán sản phẩm cho khách hàng cụ thể thông qua biến và cấu trúc dữ liệu map được khởi tạo trước.
- Để tính tổng doanh thu của công ty trong khoảng thời gian, ta quy thời gian về giây thông qua hàm timeToSec và sử dụng lower_bound, upper_bound để tính doanh thu trong khoảng thời gian đó.

Mã nguồn C++:

```
#include <bits/stdc++.h>
using namespace std;
long long timeToSec(const string &s) {
    int h = stoi(s.substr(0, 2));
    int m = stoi(s.substr(3, 2));
    int sec = stoi(s.substr(6, 2));
    return h * 3600 + m * 60 + sec;
}
int main() {
    string customer, product, shop, timeStr;
    long long price;
    long long total_orders = 0;
    long long total_revenue = 0;
    unordered_map<string, long long> revenue_of_shop;
    unordered_map<string, unordered_map<string, long long>>
consume_customer_shop;
    vector<pair<long long, long long>> time_price;
    while (true) {
        if (!(cin >> customer)) return 0;
        if (customer == "#") break;
        cin >> product >> price >> shop >> timeStr;
        total_orders++;
        total_revenue += price;
```

```

        revenue_of_shop[shop] += price;
        consume_customer_shop[customer][shop] += price;
        time_price.push_back({timeToSec(timeStr), price});
    }
    sort(time_price.begin(), time_price.end());
    vector<long long> prefix(time_price.size() + 1, 0);
    for (int i = 0; i < (int)time_price.size(); i++)
        prefix[i + 1] = prefix[i] + time_price[i].second;
    string query;
    while (true) {
        cin >> query;
        if (query == "#") break;
        if (query == "?total_number_orders") {
            cout << total_orders << "\n";
        }
        else if (query == "?total_revenue") {
            cout << total_revenue << "\n";
        }
        else if (query == "?revenue_of_shop") {
            string s;
            cin >> s;
            cout << revenue_of_shop[s] << "\n";
        }
        else if (query == "?total_consume_of_customer_shop") {
            string cus, sh;
            cin >> cus >> sh;
            cout << consume_customer_shop[cus][sh] << "\n";
        }
        else if (query == "?total_revenue_in_period") {
            string t1, t2;
            cin >> t1 >> t2;
            long long from = timeToSec(t1);
            long long to = timeToSec(t2);
            int l = lower_bound(time_price.begin(), time_price.end(),
                               make_pair(from, 0LL)) - time_price.begin();
            int r = upper_bound(time_price.begin(), time_price.end(),
                               make_pair(to, LLONG_MAX)) - time_price.begin();
            cout << (prefix[r] - prefix[l]) << "\n";
        }
    }
    return 0;
}

```

Chương 3: Xây dựng ứng dụng đơn giản

Link GitHub: <https://github.com/nguyenLTpk/ProjectI-IT3150.git>

1. Mô tả bài toán, đầu vào và đầu ra

Đây là một ứng dụng Web giải quyết bài toán kinh điển Traveling Salesperson Problem (TSP): Tìm lộ trình ngắn nhất đi qua tất cả các điểm đã chọn một lần duy nhất và quay trở lại điểm xuất phát.

- Đầu vào (Input):
 - Người dùng tương tác trực tiếp trên bản đồ số (OpenStreetMap).
 - Tập hợp các tọa độ GPS được tạo ra khi người dùng click chuột vào bản đồ.
 - Điểm đầu tiên (đánh số 1) được mặc định là điểm xuất phát và kết thúc.
- Đầu ra (Output):
 - Thứ tự di chuyển: Danh sách các điểm đã được sắp xếp lại sao cho tổng quãng đường là ngắn nhất.
 - Lộ trình trực quan: Đường đi (Route) được vẽ màu xanh trên bản đồ, bám theo đường giao thông thực tế.
 - Chi tiết: Bảng liệt kê từng chặng (Ví dụ: Từ điểm 1 -> điểm 4: 5.2km) và tổng độ dài quãng đường.

2. Mô tả lõi thuật toán

Backend được viết bằng Java, xử lý việc tính toán phức tạp.

- Thu thập dữ liệu khoảng cách:
 - Hệ thống ưu tiên sử dụng OSRM API (Open Source Routing Machine) để lấy ma trận khoảng cách thực tế (lái xe ô tô) giữa tất cả các cặp điểm.
 - Cơ chế Fallback: Nếu API bị lỗi hoặc quá tải (hoặc số lượng điểm quá lớn), hệ thống sẽ tự động chuyển sang tính toán khoảng cách đường chim bay bằng công thức Haversine (tính khoảng cách trên mặt cầu).
- Thuật toán tối ưu hóa:
 - Giải thuật chính xác: Với số lượng điểm nhỏ, hệ thống dùng đệ quy quay lui để tìm ra kết quả tối ưu tuyệt đối.
 - Giải thuật tham lam: Được sử dụng để tìm lời giải nhanh hoặc làm cận trên cho thuật toán chính xác.

→ Kết quả trả về là con đường ngắn nhất tìm được

3. Thiết kế hệ thống và Giao diện

Kiến trúc

- Mô hình: Client-Server thông qua giao thức WebSocket để đảm bảo tính thời gian thực (Real-time).
- Giao thức: Khi người dùng ấn "Tìm đường", client gửi JSON chứa danh sách tọa độ qua WebSocket (ws://.../msg). Server xử lý và gửi trả lại tiến độ (Status) và kết quả cuối cùng (Result).

Giao diện người dùng (Frontend)

Frontend được xây dựng bằng HTML5, CSS3 và JavaScript, sử dụng thư viện Leaflet.js.

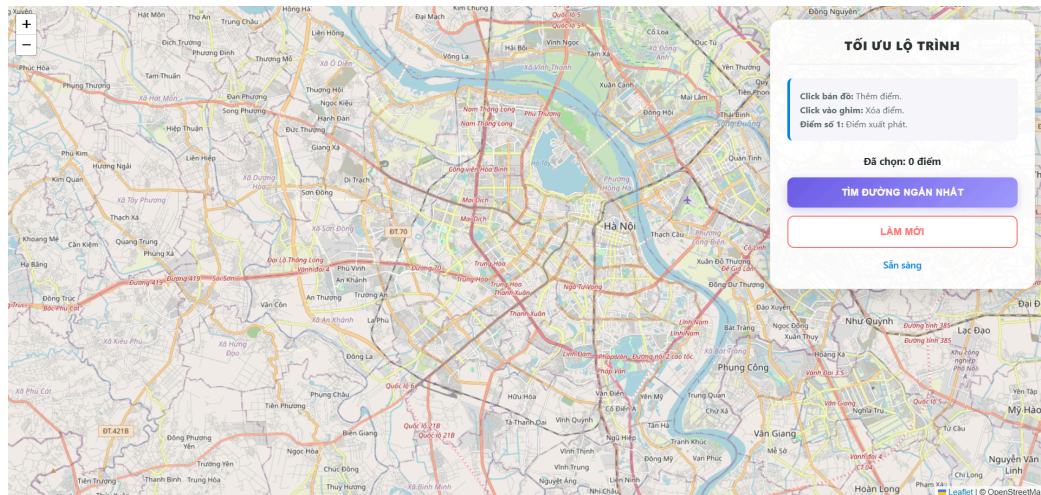
- Bản đồ nền: OpenStreetMap hiển thị full màn hình.
- Bảng điều khiển:
 - Nằm trôi nổi ở góc phải trên.
 - Hiển thị hướng dẫn sử dụng, số điểm đã chọn.
 - Nút chức năng: "TÌM ĐƯỜNG NGẮN NHẤT" và "LÀM MỚI" .
- Hệ thống Marker:
 - Waiting (Đỏ cam): Các điểm chưa được sắp xếp.
 - Start (Xanh lá): Điểm xuất phát,
 - Result (Xanh dương): Các điểm đã được đánh số lại theo lộ trình tối ưu.
- Hiển thị kết quả:
 - Khi có kết quả, Frontend gọi API OSRM client-side để vẽ đường đi chi tiết (polyline) màu xanh dương đậm nối các điểm theo thứ tự server trả về.
 - Danh sách chi tiết từng chặng hiện ra trong bảng điều khiển.

Các API chính

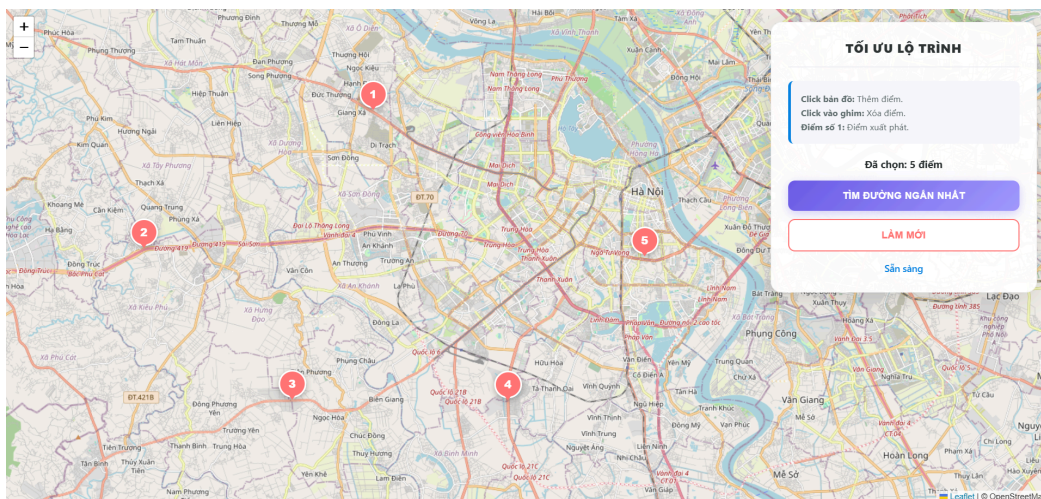
1. WebSocket Endpoint (/msg):
 - *Request*: { locations: [{lat, lng}, ...], energy: {...} }
 - *Response (Status)*: { type: "STATUS", message: "Đang tính toán..." }
 - *Response (Result)*: { type: "RESULT", path: [0, 3, 1, 2, 0], ... }.
2. External API (OSRM):
 - Backend gọi để lấy ma trận khoảng cách (Matrix API).
 - Frontend gọi để lấy hình dạng đường đi (Route/Geometry API) để vẽ lên bản đồ.

4. Kết quả mô phỏng

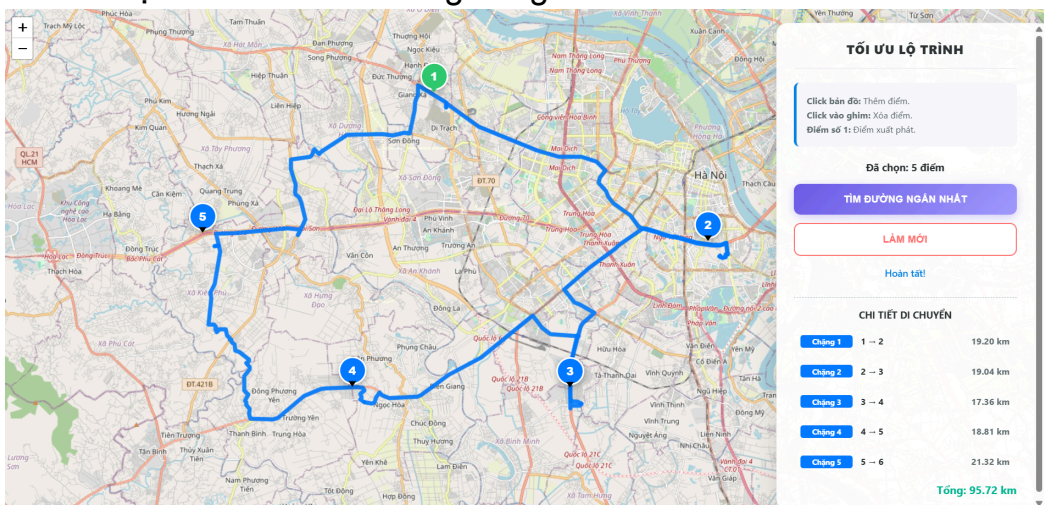
- Giao diện trước khi chọn địa điểm



- Giao diện sau khi chọn địa điểm



- Giao diện sau khi tìm đường đi ngắn nhất



→ Sau khi bấm nút, các điểm sẽ được nối với nhau bằng một đường màu xanh dương chạy dọc theo các tuyến phố thực tế (không phải đường thẳng), tạo thành một vòng khép kín.

Các điểm ghim sẽ thay đổi số thứ tự (1, 2, 3, 4) tương ứng với trình tự để tạo thành đường đi ngắn nhất.